

SI100B Project: Design and Implementation of a miniCPU

SI100B EE Part TA Team

December 16, 2025

Contents

Contents	I
1 Introduction	1
2 ISA of miniCPU	2
2.1 List of Abbreviations	2
2.2 R-type Instructions (Register)	2
2.3 I-type Instructions (Immediate)	3
2.4 S-type Instructions (Store)	4
2.5 B-type Instructions (Branch)	5
2.6 J-type Instructions (Jump)	6
3 Architecture of miniCPU	7
3.1 Main Module Descriptions	8
3.1.1 Register File	8
3.1.2 Program Counter (PC)	8
3.1.3 Arithmetic Logic Unit (ALU)	9
3.1.4 Data Memory	10
3.1.5 Immediate Extender	10
3.1.6 Controller	11
4 Requirements	15
4.1 Implementation Details	15
4.1.1 Required Tasks	15
4.1.2 Bonus Tasks	16
4.1.3 Example Results	16
4.1.4 Grading and Notes	17
4.1.5 Submission Suggestions	17
4.2 Report	18
4.3 Submission	19
4.3.1 Submission Requirements	19
4.3.2 Submission Deadline	19
4.4 Face-To-Face Check	19

1 Introduction

This project aims to design and implement a simplified central processing unit (miniCPU), covering both its datapath and controller. Through this project, students will gain a deep understanding of core computer architecture concepts, including the register file, arithmetic logic unit (ALU), program counter, instruction memory, and data memory, as well as how instruction decoding and control signal generation enable instruction execution.

In computer architecture, the terms “architecture” and “microarchitecture” refer to the instruction set architecture (ISA) and the processor implementation, respectively. The ISA specifies which instructions the processor must support, while the microarchitecture describes the hardware circuits that implement those instructions. Popular ISAs include x86, ARM, MIPS, and RISC-V. The same ISA can have multiple hardware implementations (e.g., both Intel and AMD produce x86 processors that run the same programs but have different internal circuits). This project focuses on the RISC-V ISA and processor, a rapidly growing open-source alternative to proprietary ISAs like x86 and ARM.

This is a collaborative project! Pair programming is encouraged, as it may be an effective way to complete the project.

2 ISA of miniCPU

2.1. List of Abbreviations

Table 2.1: List of Abbreviations

Abbreviation	Description
<code>rs1</code>	Register <code>rs1</code>
<code>rs2</code>	Register <code>rs2</code>
<code>rd</code>	Register <code>rd</code>
<code>pc</code>	Register <code>pc</code>
<code>imm</code>	Immediate
<code>SE()</code>	Sign Extend
<code>ZE()</code>	Zero Extend
<code>[]</code>	Memory Access
<code>lab</code>	Label
<code>f3</code>	Funct3
<code>f7</code>	Funct7
<code>opcode</code>	Opcode

In this project, you will implement a subset of the RISC-V RV32-I instruction set, consisting of 13 instructions in three categories: arithmetic, memory access, and control flow. These instructions cover basic computation, data transfer, and program control, sufficient for building simple programs.

2.2. R-type Instructions (Register)

R-type instructions operate on registers (operands: `rs1`, `rs2`, `rd`). The result of the operation on `rs1` and `rs2` is written to `rd`. The required R-type instructions are shown in table 2.2.

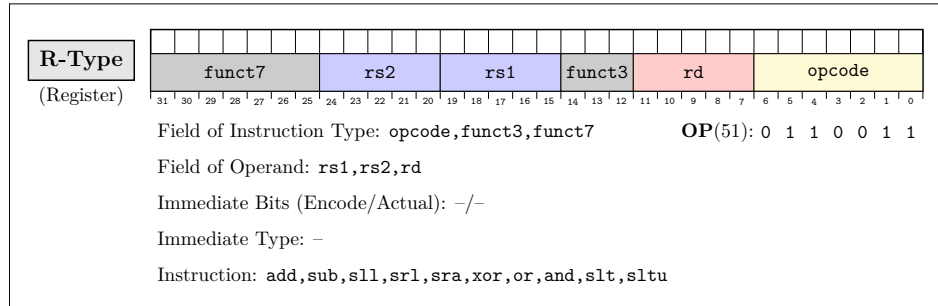


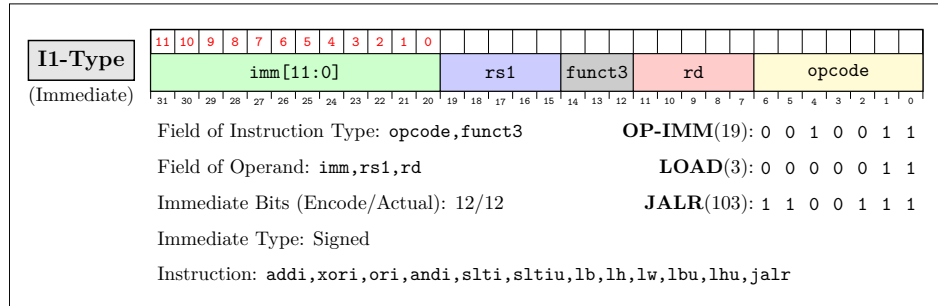
Figure 2.1: R-type Instruction Encoding

Table 2.2: R-type Instruction List

Instruction	Description	Type	opcode	f3	f7
add rd,rs1,rs2	rd=rs1+rs2	R	0110011	000	0000000
Add					Register addition
sub rd,rs1,rs2	rd=rs1-rs2	R	0110011	000	0100000
Sub					Register subtraction
and rd,rs1,rs2	rd=rs1&rs2	R	0110011	111	0000000
And					Register AND
or rd,rs1,rs2	rd=rs1 rs2	R	0110011	110	0000000
Or					Register OR
slt rd,rs1,rs2	rd=(rs1<rs2)	R	0110011	010	0000000
Set Less Than					Register set if less (signed)

2.3. I-type Instructions (Immediate)

I-type instructions use an immediate value (operands: imm, rs1, rd). These include immediate arithmetic and load instructions. The required I-type instructions are shown in table 2.3.



(a) I1-type Instruction Encoding

Figure 2.2: I-type Instruction Encoding

Table 2.3: I-type Instruction List

Instruction	Description	Type	opcode	f3	f7
addi rd,rs1,imm	rd=rs1+SE(imm)	I	0010011	000	-----
Add Immediate				Immediate addition	
andi rd,rs1,imm	rd=rs1&SE(imm)	I	0010011	111	-----
And Immediate				Immediate AND	
ori rd,rs1,imm	rd=rs1 SE(imm)	I	0010011	110	-----
Or Immediate				Immediate OR	
slti rd,rs1,imm	rd=(rs1<SE(imm))	I	0010011	010	-----
Set Less Than Immediate				Immediate set if less (signed)	
lw rd,imm(rs1)	rd=[rs1+SE(imm)]	I	0000011	010	-----
Load Word				Load word	

2.4. S-type Instructions (Store)

S-type instructions store data (operands: imm, rs1, rs2). These write the value of rs2 to the address imm(rs1). The required S-type instruction is shown in table 2.4.

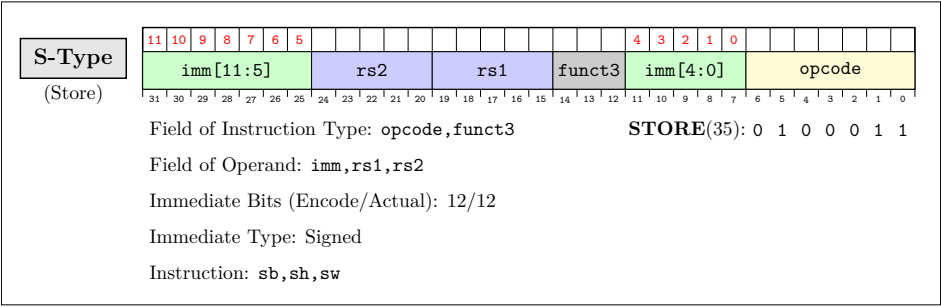


Figure 2.3: S-type Instruction Encoding

Table 2.4: S-type Instruction List

Instruction	Description	Type	opcode	f3	f7
sw rs2,imm(rs1)	[rs1+SE(imm)]=rs2	S	0100011	010	-----
Store Word				Store word	

2.5. B-type Instructions (Branch)

B-type instructions are for conditional branches (operands: imm, rs1, rs2). These compare rs1 and rs2 and branch if the condition is met. The required B-type instruction is shown in table 2.5.

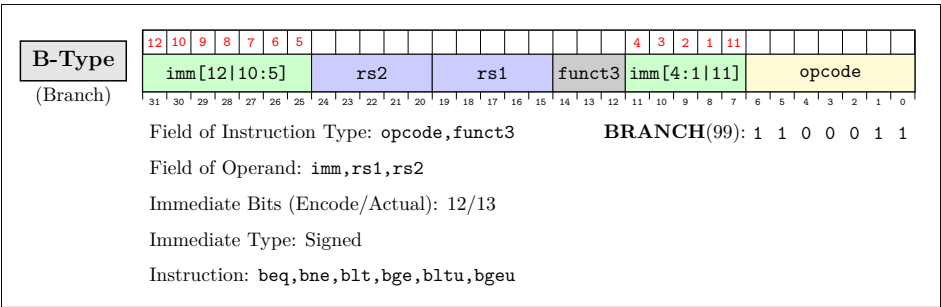


Figure 2.4: B-type Instruction Encoding

Table 2.5: B-type Instruction List

Instruction	Description	Type	opcode	f3	f7
bne rs1,rs2,lab	if(rs1!=rs2) pc+=SE(imm,1'b0)	B	1100011	001	-----
Branch If Not Equal				Branch if not equal	

2.6. J-type Instructions (Jump)

J-type instructions are for unconditional jumps and link (operands: `imm`, `rd`). The required J-type instruction is shown in table 2.6.

Table 2.6: J-type Instruction List

Instruction	Description	Type	opcode	f3	f7
j al rd,imm	rd=pc+4, pc=pc+SE(imm)	J	1101111	000	-----
Jump and Link				Jump and link	

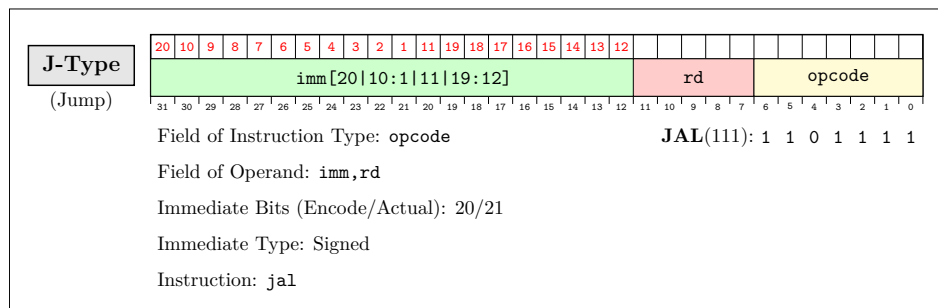


Figure 2.5: J-type Instruction Encoding

3 Architecture of miniCPU

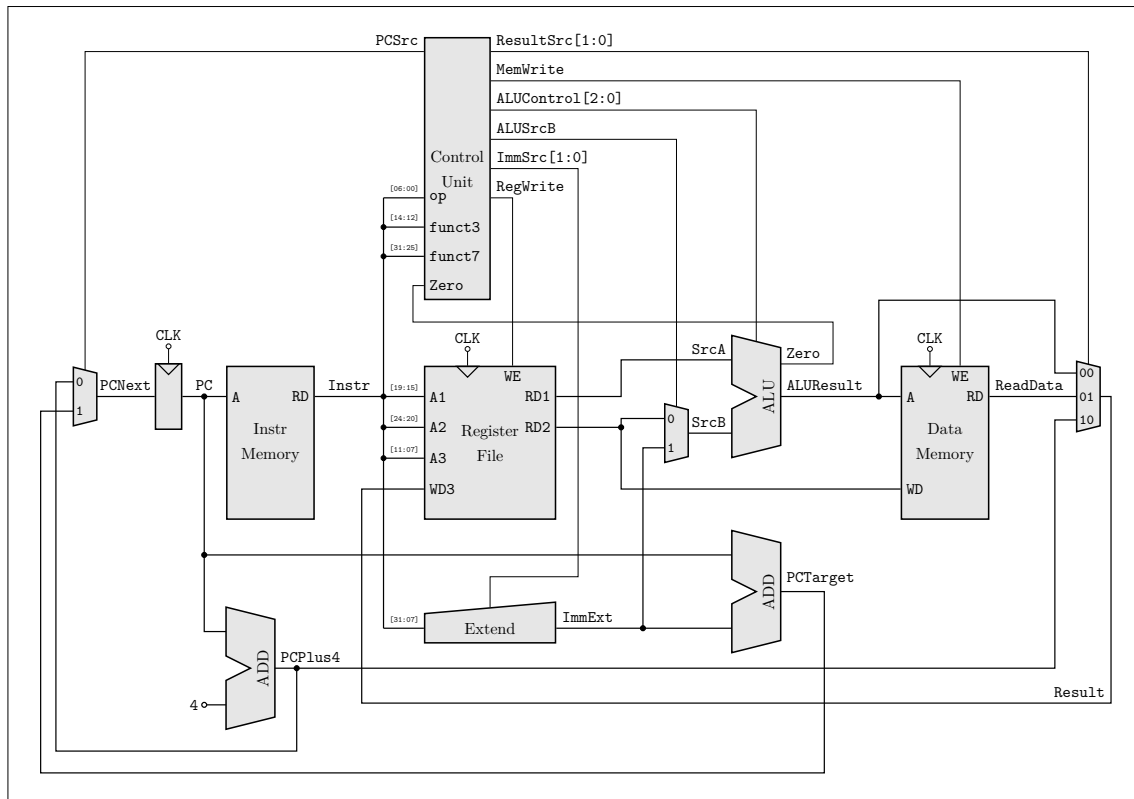


Figure 3.1: Overall Architecture of miniCPU

The miniCPU uses a classic five-stage structure: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). In this project, you will implement a single-cycle CPU, where each instruction completes all five stages in one clock cycle. Figure 3.1 shows the main components and their connections.

3.1. Main Module Descriptions

3.1.1 Register File

The register file is the core storage component of the CPU. According to the datapath diagram, it contains 32 general-purpose 32-bit registers.

Key features:

- Two read ports (A1/RD1, A2/RD2) and one write port (A3/WD3).
- Read is combinational logic; write occurs on the rising clock edge when write enable (WE) is asserted.
- Register x0 is always zero.

Interface definition is shown in tables 3.1 and 3.2.

Table 3.1: Register File Input Signals

Input	Width	Description
CLK	1	Clock signal
RST	1	Reset signal (asynchronous)
WE	1	Write enable (from Control Unit's <code>RegWrite</code>)
A1	5	Address of source register <code>rs1</code> (<code>Instr[19:15]</code>)
A2	5	Address of source register <code>rs2</code> (<code>Instr[24:20]</code>)
A3	5	Address of destination register <code>rd</code> (<code>Instr[11:07]</code>)
WD3	32	Data of destination register <code>rd</code> data (from <code>ResultMux</code>)

Table 3.2: Register File Output Signals

Output	Width	Description
RD1	32	Data of source register <code>rs1</code> (to <code>SrcA</code>)
RD2	32	Data of source register <code>rs2</code> (to <code>SrcBMux</code> and <code>DataMemory</code>)

3.1.2 Program Counter (PC)

The PC is a 32-bit register that stores the current instruction address.

Key features:

- Updates on the rising clock edge, latching the value of `PCNext` to `PC`.

- **PCNext** is selected externally: either sequential **PCPlus4** or a jump target **PCTarget**.

Interface definition is shown in tables 3.3 and 3.4.

Table 3.3: PC Input Signals

Input	Width	Description
CLK	1	Clock signal
RST	1	Reset signal (asynchronous)
PCNext	32	Next instruction address (from PCMux)

Table 3.4: PC Output Signals

Output	Width	Description
PC	32	Current instruction address

3.1.3 Arithmetic Logic Unit (ALU)

The ALU performs arithmetic and logic operations.

Key features:

- **Arithmetic Operations:** Performs addition (**add/addi**) and subtraction (**sub**), computing **SrcA + SrcB** or **SrcA - SrcB**. Addition is also used for address calculation in load/store instructions (**lw/sw**).
- **Logical Operations:** Executes bitwise AND (**and/andi**) and OR (**or/ori**) operations on the two operands.
- **Comparison:** Implements set-less-than (**slt/slti**) by performing signed comparison. If **SrcA < SrcB** (treating both as signed integers), the result is 1; otherwise, 0.
- **Zero Flag Generation:** Produces a **Zero** output signal that is asserted (1) when **ALUResult** equals zero. This flag is critical for conditional branch instructions (e.g., **beq/bne**) to determine whether to take the branch.
- **Operation Selection:** The 3-bit **ALUControl** signal selects the operation: 000 (Add), 001 (Sub), 010 (And), 011 (Or), 101 (Slt).

Interface definition is shown in tables 3.5 and 3.6.

Table 3.5: ALU Input Signals

Input	Width	Description
SrcA	32	Operand A (from RD1)
SrcB	32	Operand B (from RD2 or ImmExt , based on SrcBMux)
ALUControl	3	ALU operation control

Table 3.6: ALU Output Signals

Output	Width	Description
ALUResult	32	Alu operation result
Zero	1	Zero flag (set to 1 if ALUResult is zero)

3.1.4 Data Memory

Data memory is used for data segment access.

Key features:

- Read is combinational (output changes with address).
- Write is controlled by **WE** and occurs on the rising clock edge.

Interface definition is shown in tables 3.7 and 3.8.

Table 3.7: Data Memory Input Signals

Input	Width	Description
CLK	1	Clock signal
RST	1	Reset signal (asynchronous)
WE	1	Write enable (from MemWrite)
A	32	Memory address (from ALUResult)
WD	32	Write data to memory (from RD2)

Table 3.8: Data Memory Output Signals

Output	Width	Description
RD	32	Read data from memory

3.1.5 Immediate Extender

The Immediate Extender extracts the immediate value from the instruction and sign-extends it to 32 bits based on the instruction type.

Key features:

- Extracts immediate fields from `Instr` based on `ImmSrc`.
- Performs sign extension to generate a 32-bit `ImmExt`.

Interface definition is shown in ?? and??.

Table 3.9: Immediate Extender Input Signals

Input	Width	Description
<code>Instr</code>	32	Current instruction (from <code>InstructionMemory</code>)
<code>ImmSrc</code>	2	Immediate type select (from Controller)

Table 3.10: Immediate Extender Output Signals

Output	Width	Description
<code>ImmExt</code>	32	Extended immediate value (to <code>SrcBMux</code> and <code>PCTarget</code>)

3.1.6 Controller

The controller is the command center of the miniCPU, responsible for decoding instructions and generating all necessary control signals to coordinate the operation of the datapath. It consists of two main components: the **Main Decoder** and the **ALU Decoder**.

Key features:

- **Instruction Decoding:** Analyzes the opcode field (`Instr[6:0]`) to determine the instruction type (R-type, I-type, S-type, B-type, or J-type). Uses `funct3` (`Instr[14:12]`) and `funct7` (`Instr[31:25]`) fields for further differentiation within each instruction type.
- **Datapath Control Signal Generation:** Produces control signals that direct multiplexers, enable signals, and operation codes throughout the datapath:
 - **RegWrite:** Enables writing to the register file when asserted.
 - **MemWrite:** Enables writing to data memory for store instructions.
 - **ALUSrcB:** Selects whether the ALU's second operand comes from `RD2` (register) or `ImmExt` (immediate).
 - **ResultSrc:** Determines the source of data to write back to the register file (ALU result, memory data, or `PC+4`).

- **ALU Operation Control:** The ALU Decoder combines `opcode`, `funct3`, and `funct7` to generate the 3-bit `ALUControl` signal, specifying which operation the ALU should perform (add, subtract, AND, OR, or set-less-than).
- **Immediate Type Selection:** Generates the 2-bit `ImmSrc` signal to instruct the Immediate Extender on how to extract and extend the immediate value based on instruction type.
- **Branch and Jump Logic:** Combines the `Zero` flag from the ALU with branch/jump instruction information to generate `PCSrc`, which determines whether the next PC should be sequential (`PC+4`) or a computed target address (`PCTarget`). For `bne` instructions, the branch is taken when `Zero` is low (registers are not equal); for `jal`, the branch is unconditional.

Control Path Supporting Modules

Several small combinational modules work alongside the controller to facilitate datapath operations:

1. PCPlus4Adder

- **Function:** Computes the address of the next sequential instruction.
- **Inputs:** PC (32-bit, current program counter value)
- **Outputs:** `PCPlus4` (32-bit, computed as `PC + 4`)
- **Usage:** Provides the default next instruction address for non-branching instructions and serves as the return address for `jal`.

2. PCTargetAdder

- **Function:** Calculates the target address for branch and jump instructions.
- **Inputs:** PC (32-bit), `ImmExt` (32-bit, sign-extended immediate)
- **Outputs:** `PCTarget` (32-bit, computed as `PC + ImmExt`)
- **Usage:** Used for relative addressing in branch (`bne`) and jump (`jal`) instructions.

3. PCMux (PC Source Multiplexer)

- **Function:** Selects the next program counter value.
- **Inputs:** PCPlus4 (32-bit), PCTarget (32-bit), PCSrc (1-bit control signal)
- **Outputs:** PCNext (32-bit)
- **Operation:** When PCSrc = 0, outputs PCPlus4; when PCSrc = 1, outputs PCTarget.
- **Usage:** Implements control flow by selecting between sequential execution and branch/jump targets.

4. SrcBMux (ALU Source B Multiplexer)

- **Function:** Selects the second operand for the ALU.
- **Inputs:** RD2 (32-bit, register data), ImmExt (32-bit, extended immediate), ALUSrcB (1-bit control signal)
- **Outputs:** SrcB (32-bit)
- **Operation:** When ALUSrcB = 0, outputs RD2 (for R-type instructions); when ALUSrcB = 1, outputs ImmExt (for I-type instructions and address calculations).

5. ResultMux (Result Source Multiplexer)

- **Function:** Selects the data to be written back to the register file.
- **Inputs:** ALUResult (32-bit), ReadData (32-bit, from data memory), PCPlus4 (32-bit), ResultSrc (2-bit control signal)
- **Outputs:** Result (32-bit, connected to WD3 of register file)
- **Operation:**
 - ResultSrc = 00: outputs ALUResult (for arithmetic/logic instructions)
 - ResultSrc = 01: outputs ReadData (for load instructions)
 - ResultSrc = 10: outputs PCPlus4 (for jal instruction to save return address)

Interface definition is shown in tables 3.11 and 3.12.

Table 3.11: Controller Input Signals

Input	Width	Description
<code>op</code>	7	Opcode (<code>Instr[6:0]</code>)
<code>funct3</code>	3	3-bit function code (<code>Instr[14:12]</code>)
<code>funct7</code>	7	7-bit function code (<code>Instr[31:25]</code>)
<code>Zero</code>	1	Zero flag from ALU

Table 3.12: Controller Output Signals

Output	Width	Description
<code>PCSrc</code>	1	PC source select (0: <code>PCPlus4</code> , 1: <code>PCTarget</code>)
<code>ResultSrc</code>	2	Write-back data select (00: <code>ALUResult</code> , 01: <code>ReadData</code> , 10: <code>PCPlus4</code>)
<code>MemWrite</code>	1	Data memory write enable
<code>ALUControl</code>	3	ALU operation (000: <code>Add</code> , 001: <code>Sub</code> , 010: <code>And</code> , 011: <code>Or</code> , 101: <code>Slt</code>)
<code>ALUSrcB</code>	1	ALU operand B select (0: <code>RD2</code> , 1: <code>ImmExt</code>)
<code>ImmSrc</code>	2	Immediate extension type (00: <code>I</code> , 01: <code>S</code> , 10: <code>B</code> , 11: <code>J</code>)
<code>RegWrite</code>	1	Register file write enable

4 Requirements

4.1. Implementation Details

This section provides implementation details and task breakdown for project development and report writing. Each task is weighted for grading; please clearly indicate completed and planned parts in your implementation and report.

4.1.1 Required Tasks

1. Implement the miniCPU datapath: including register file, ALU, program counter (PC), instruction memory (IMem), data memory (DMem), with functional implementation and interface definition. (20%)
2. Implement the miniCPU controller: including instruction decoding, control signal generation, connection to the datapath, and control path verification to ensure correct instruction execution. (20%)
3. Write a simple assembler in Python: convert RISC-V assembly text to machine code (binary/hex image) for loading into instruction memory for simulation or testing. (20%)
4. Manually write and load assembly programs: create several assembly test programs to implement specified functions, convert them to machine code, load into IMem, and observe CPU behavior (register/memory state, PC changes, etc.). This is mainly for functional verification and debugging. (15%)
5. Project report: provide a systematic written report explaining your implementation, key design decisions, test cases and results, and problems encountered and solved. (25%)

4.1.2 Bonus Tasks

The Bonus section is optional. Completing the Bonus can earn you up to an additional 25% extra credit.

1. Implement additional RISC-V instructions beyond the required
 - (a) Each additional instruction implemented correctly: 0.4% per instruction
 - (b) Complete implementation of a full instruction category (e.g., R-type, I-type, etc.): 2.5% per category
 - (c) Complete implementation of the full RV32-I instruction set: 15%
 - (d) Implement additional instruction set extensions (e.g., RV32-F, RV32-V, etc.): 25%
2. Implement a multi-cycle miniCPU: Depending on the design, completing any full multi-cycle design, resolving associated hazards, and verifying correctness can earn 20% extra credit. A complete five-stage pipelined implementation can earn 25% extra credit.
3. Implement the miniCPU using another hardware description language (e.g., SpinalHDL): Correct completion earns 25% extra credit.
4. Excellent report: Use $\text{\LaTeX}$ to produce the report. If the report is complete, well-structured, illustrated with images, and beautifully formatted, up to 2% extra credit can be earned based on quality.
5. Other innovative extensions: such as adding exception/interrupt handling, caching mechanisms, branch prediction, etc. Depending on the complexity and completeness of the implementation, up to 25% extra credit can be earned.

4.1.3 Example Results

In your program's report, you should provide detailed information about your test cases, expected results, and actual test results, including: the assembly code of the test cases, the expected register/memory states, and screenshots or waveform diagrams of the test results.

4.1.4 Grading and Notes

- Bonus tasks and testing requirements: Completing bonus tasks can earn extra points and will be included in the overall evaluation. All implementations must provide sufficient test cases to fully demonstrate correctness.
- Face-to-face check: Performance during the check session will affect the final score. If performance is poor (e.g., completely unable to explain your implementation), points may be deducted from the original score.
- Error analysis and process demonstration: Even if there are errors in the implementation, as long as you provide a detailed analysis of the causes, demonstrate the debugging process, and propose clear improvement ideas in the report, you can still earn most of the points based on completeness and depth of technical understanding.
- Tool usage and academic integrity: You are allowed to use any AI tools for assistance, but all submitted content must ensure that you or your team members can thoroughly understand and independently explain it. If you reference any public resources (including open-source code), you must clearly indicate the source and specific modifications in the report. Any form of inter-group plagiarism is strictly prohibited.

4.1.5 Submission Suggestions

- The submission package should include source code, assembler, sample assembly programs, generated machine code images, run/simulation instructions (with key command line examples), and the final report (preferably in L^AT_EX).
- For key modules in the report, provide clear diagrams (circuit or datapath), control signal explanations, immediate extension and memory interface definitions, test cases, and verification screenshots or waveforms.

4.2. Report

You must complete a well-formatted report. Avoid submitting a disorganized collection of text and images; poor formatting may result in point deductions. The report should include the following:

1. CPU Implementation

- If only required tasks are completed: provide diagrams and explanations of key modules, especially those not explicitly covered in class (e.g., control signal flow, immediate extension, datapath, register file and memory interfaces).
- If any optional or extended features are implemented: describe these in detail, including design motivation, implementation details, timing/resource/performance impact, and related diagrams or flowcharts.
- Use LaTeX or Markdown for writing; diagrams should have clear titles and explanations; code should use appropriate code environments (e.g., `lstlisting`).

2. Test Cases and Verification

- Provide several representative assembly test programs that can distinguish correct from incorrect implementations; avoid trivial tests (e.g., repeatedly operating on the zero register).
- For each test case, provide expected and actual verification information, such as register state after execution, key memory contents, output logs, or simulation waveforms, to prove correctness or locate errors.
- If tests depend on specific toolchains, simulators, or steps, provide reproduction instructions in the report (including command line examples, input file locations, and environment).

The report should be well-structured, with diagrams and explanations, and include an appendix or attachment listing source code directories and build/run instructions. Missing key explanations or poor formatting may result in point deductions.

4.3. Submission

4.3.1 Submission Requirements

1. **Project code and resources:** Package all project files (CPU implementation, assembler, test programs, documentation, etc.) as a ZIP file and submit to Gradescope under “Project Code”.
2. **Project report:** Submit the report as a PDF to Gradescope under “Project Report”.

4.3.2 Submission Deadline

Submit all materials by the course deadline. Late submissions may affect your grade.

4.4. Face-To-Face Check

There will be a face-to-face check session, including:

1. Live demonstration and functional verification. You will demonstrate your CPU; the review team may run preset assembly programs and expect correct results. If not all tests pass, as long as you can accurately locate the problem and propose reasonable solutions, you will not necessarily lose points.
2. Design and implementation explanation. Be prepared to explain key modules and implementation details in your report, such as control signal generation, immediate extension, branch/jump logic, register file and memory interfaces, and answer technical questions.
3. Optional feature demonstration. If you implemented extra features (e.g., more instructions, pipelining, exception/interrupt handling), demonstrate and explain their design and impact on the architecture.
4. Team contribution statement. Briefly explain each member’s contribution for the reviewers to understand collaboration.

The check will take place after weeks 17-18, following the completion of final exams and project submissions; specific time and location will be notified separately. Please prepare in advance to ensure that someone from your team can attend for demonstration and Q & A.